고객을 세그먼테이션하자 [프로젝트] - 허재민

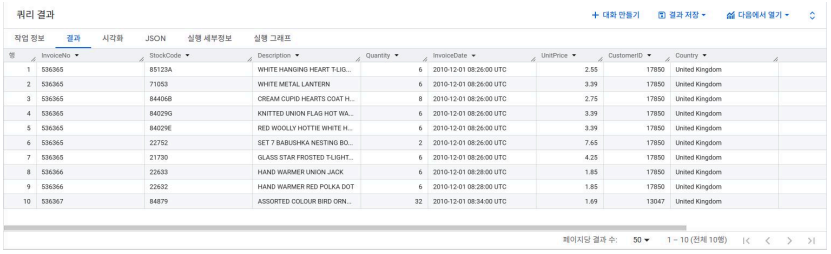
11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
LIMIT 10;
```

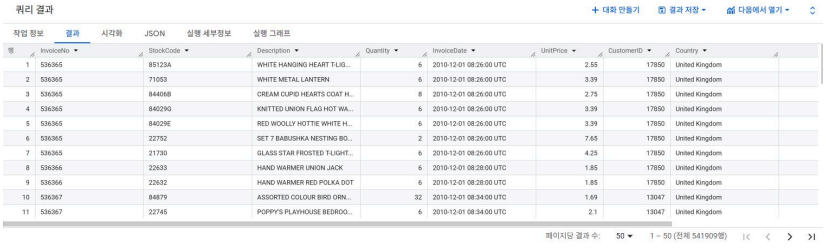
[결과 이미지를 넣어주세요]



- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

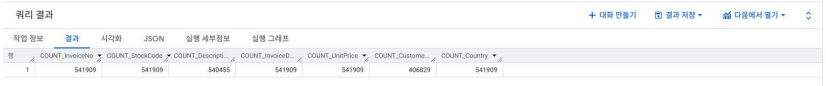


데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT COUNT(InvoiceNo) AS COUNT_InvoiceNo,
      COUNT(StockCode) AS COUNT_StockCode,
      COUNT(Description) AS COUNT_Description,
      COUNT(Quantity) AS COUNT_Quantity,
      COUNT(InvoiceDate) AS COUNT_InvoiceDate,
      COUNT(UnitPrice) AS COUNT_UnitPrice,
      COUNT(CustomerID) AS COUNT_CustomerID,
      COUNT(Country) AS COUNT_Country
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
LIMIT 1
```

[결과 이미지를 넣어주세요]



쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Descripti...	COUNT_InvoiceD...	COUNT_UnitPrice	COUNT_Custome...	COUNT_Country
1	541909	541909	540455	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perce
ntage
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
UNION ALL
SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perce
ntage
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
UNION ALL
SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_per
centage
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
UNION ALL
SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percen
tage
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
UNION ALL
SELECT
    'InvoiceDate' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_per
centage
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
UNION ALL
SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perce
ntage
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
UNION ALL
SELECT
    'CustomerID' AS column_name,
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perc
entage
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
UNION ALL
SELECT
    'Country' AS column_name,
    ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percent
age
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```
SELECT DISTINCT Description
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
WHERE StockCode = '85123A';
```

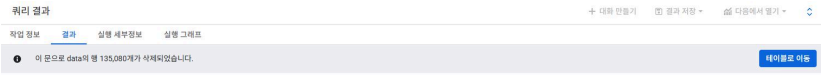
[결과 이미지를 넣어주세요]

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
DELETE FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
WHERE CustomerID IS NULL
OR Description IS NULL;
```

[결과 이미지를 넣어주세요]



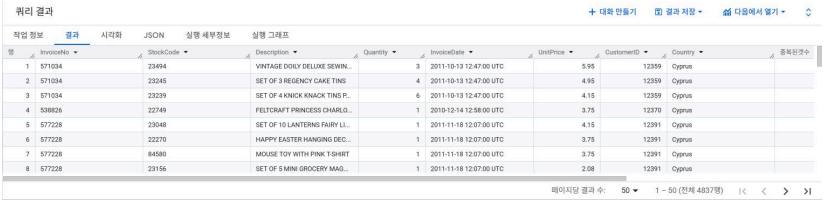
11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
SELECT InvoiceNo, StockCode, Description,Quantity, InvoiceDate, UnitPrice, CustomerID, Country, CO
UNT(*) AS `중복된갯수`
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY InvoiceNo, StockCode, Description,Quantity, InvoiceDate, UnitPrice, CustomerID, Country
HAVING COUNT(*) > 1
```

[결과 이미지를 넣어주세요]



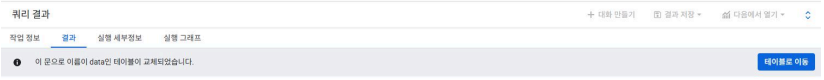
중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**

- **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.data` AS
SELECT DISTINCT *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]



11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo`의 개수를 출력하기**

```
SELECT COUNT(DISTINCT InvoiceNo)
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

- **고유한 `InvoiceNo`를 앞에서부터 100개를 출력하기**

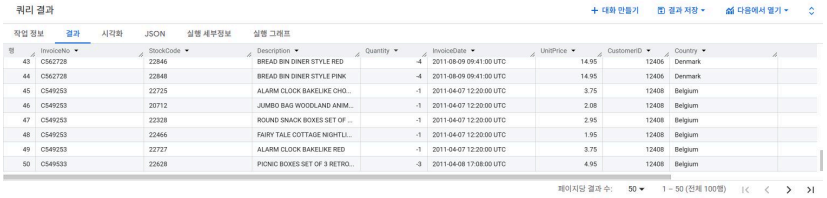
```
SELECT InvoiceNo
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY InvoiceNo
ORDER BY InvoiceNo ASC
LIMIT 100;
```

[결과 이미지를 넣어주세요]

- **`InvoiceNo`가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)**

```
SELECT *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

[결과 이미지를 넣어주세요]



쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
43	C562728	22846	BREAD BIN DINER STYLE RED	-4	2011-08-09 09:41:00 UTC	14.95	12406	Denmark
44	C562728	22848	BREAD BIN DINER STYLE PINK	-4	2011-08-09 09:41:00 UTC	14.95	12406	Denmark
45	C549253	22725	ALARM CLOCK BAKELIKE CHO...	-1	2011-04-07 12:20:00 UTC	3.75	12408	Belgium
46	C549253	20712	JUMBO BAG WOODLAND ANIM...	-1	2011-04-07 12:20:00 UTC	2.08	12408	Belgium
47	C549253	22328	ROUND SNACK BOXES SET OF ...	-1	2011-04-07 12:20:00 UTC	2.95	12408	Belgium
48	C549253	22466	FAIRY TALE COTTAGE NIGHTLI...	-1	2011-04-07 12:20:00 UTC	1.95	12408	Belgium
49	C549253	22727	ALARM CLOCK BAKELIKE RED	-1	2011-04-07 12:20:00 UTC	3.75	12408	Belgium
50	C549533	22628	PICNIC BOXES SET OF 3 RETRO...	-3	2011-04-08 17:08:00 UTC	4.95	12408	Belgium

페이지당 결과 수: 50 1 – 50 (전체 100행)

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*) * 100, 1)
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data
```

[결과 이미지를 넣어주세요]

쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	f0_
1	2.2

`StockCode` 살펴보기

- **고유한 `StockCode` 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT StockCode)
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	f0_
1	3684

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

[결과 이미지를 넣어주세요]

쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196

페이지당 결과 수: 50 1 – 10 (전체 10행)

- **`StockCode` 의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
)
WHERE number_count < 5
```

[결과 이미지를 넣어주세요]

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

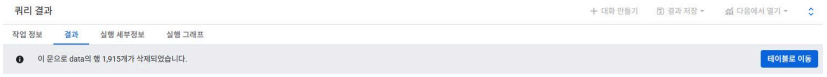
```
SELECT
  ROUND(
    COUNTIF(
      LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) < 5
     ) / COUNT(*) * 100,
     2
   ) AS StockCode_A_Ratio
  FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
WHERE LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) <= 1;
```

[결과 이미지를 넣어주세요]



`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY Description
```

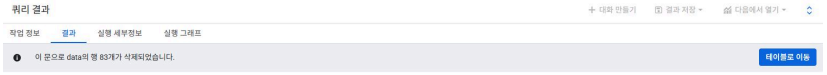
```
ORDER BY description_cnt DESC
LIMIT 30;
```

[결과 이미지를 넣어주세요]

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
WHERE Description = 'High Resolution Image'
OR Description = 'Next Day Carriage';
```

[결과 이미지를 넣어주세요]



- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.data` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

`UnitPrice` 살펴보기

- `UnitPrice`의 최솟값, 최댓값, 평균을 구하기

```
SELECT  MIN(UnitPrice) AS min_price, MAX(UnitPrice) AS max_price, AVG(UnitPrice) AS avg_price
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

- **단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기**

```
SELECT COUNT(Quantity) AS cnt_quantity, MIN(Quantity) AS min_quantity, MAX(Quantity) AS max_quanti
ty, AVG(Quantity) AS avg_quantity
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
WHERE Unitprice = 0;
```

[결과 이미지를 넣어주세요]

- `UnitPrice = 0` **를 제거하고 일관된 데이터셋을 유지하기**

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.data` AS
SELECT *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
WHERE Unitprice != 0;
```

[결과 이미지를 넣어주세요]

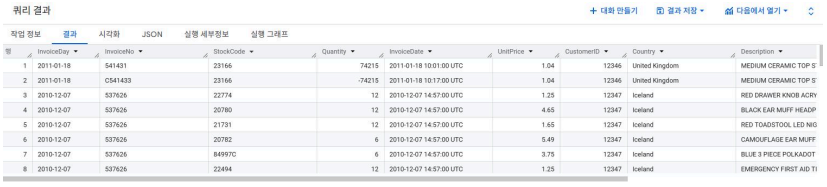
11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT DATE(InvoiceDate) AS InvoiceDay, *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]



- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

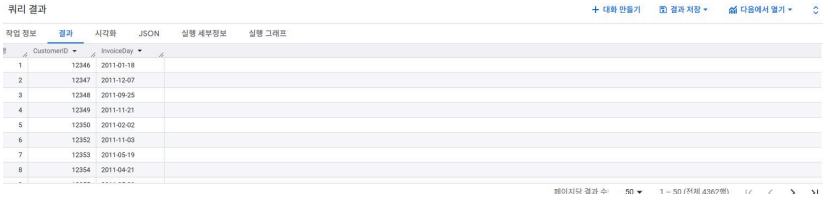
```
SELECT
   MAX(DATE(InvoiceDate)) AS most_recent_date,
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

[결과 이미지를 넣어주세요]

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY CustomerID
```

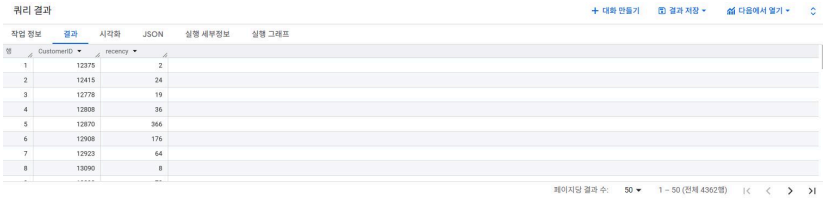
[결과 이미지를 넣어주세요]



- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM project_name.modulabs_project.data
  GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

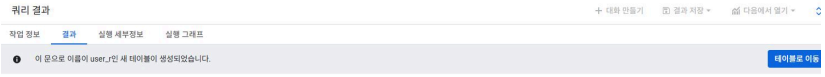


- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.user_r` AS
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
```

```
  GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

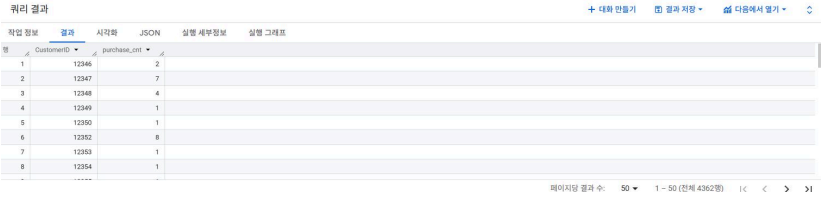


Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY CustomerID
```

[결과 이미지를 넣어주세요]



- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY CustomerID
```

[결과 이미지를 넣어주세요]

- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.user_rf` AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(InvoiceNo) AS purchase_cnt
  FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
  GROUP BY CustomerID
```

```
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
  GROUP BY CustomerID
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN `project-23e56097-5f6c-4899-bd8.modulabs_project.user_r` AS ur
  ON pc.CustomerID = ur.CustomerID;
```

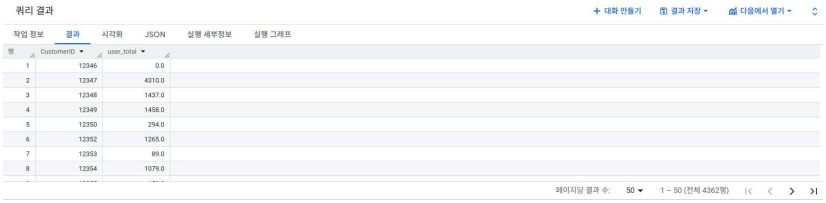
[결과 이미지를 넣어주세요]

Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice),0) AS user_total
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
GROUP BY CustomerID
```

[결과 이미지를 넣어주세요]

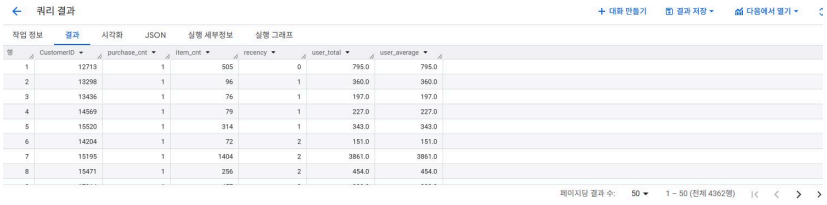


- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
```

```
  rf.recency,
  ut.user_total,
   ROUND(ut.user_total / rf.purchase_cnt,0) AS user_average
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.user_rf` rf
LEFT JOIN (
  -- 고객 별 총 지출액
  SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice),0) AS user_total
    FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
    GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;
```

[결과 이미지를 넣어주세요]

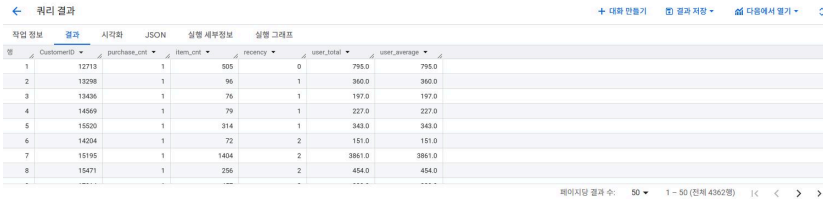


RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.user_rfm`
```

[결과 이미지를 넣어주세요]



11-8. 추가 Feature 추출

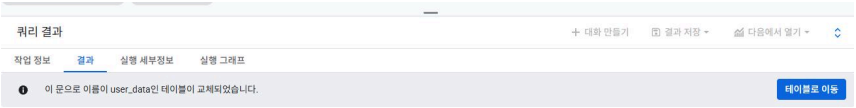
1. 구매하는 제품의 다양성

- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
  GROUP BY CustomerID
)
```

```
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

[결과 이미지를 넣어주세요]

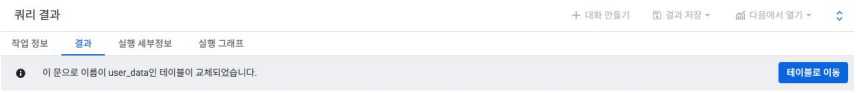


2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합**

```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.user_data` AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END AS average_in
terval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate), DA
Y) AS interval_
    FROM
      `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

[결과 이미지를 넣어주세요]



3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
    (취소 비율은 소수점 두번째 자리)

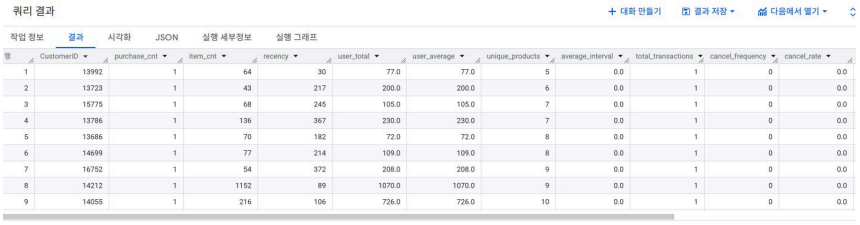
```
CREATE OR REPLACE TABLE `project-23e56097-5f6c-4899-bd8.modulabs_project.user_data` AS

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency
  FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.data`
  GROUP BY CustomerID
)

SELECT u.*, t.* EXCEPT(CustomerID), t.cancel_frequency / t.total_transactions AS cancel_rate
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.customerID;
```

[결과 이미지를 넣어주세요]

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
SELECT *
FROM `project-23e56097-5f6c-4899-bd8.modulabs_project.user_data`
```

[결과 이미지를 넣어주세요]



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	13992	1	64	30	77.0	77.0	5	0.0	1	0	0.0
2	13723	1	43	217	200.0	200.0	6	0.0	1	0	0.0
3	15775	1	68	245	105.0	105.0	7	0.0	1	0	0.0
4	13786	1	136	367	230.0	230.0	7	0.0	1	0	0.0
5	13686	1	70	182	72.0	72.0	8	0.0	1	0	0.0
6	14699	1	77	214	109.0	109.0	8	0.0	1	0	0.0
7	16752	1	54	372	208.0	208.0	9	0.0	1	0	0.0
8	14212	1	1152	89	1070.0	1070.0	9	0.0	1	0	0.0
9	14055	1	216	106	726.0	726.0	10	0.0	1	0	0.0

회고

[회고 내용을 작성해주세요]

Keep :

함수	역할
`REGEXP_REPLACE()`	특정 문자 제거
`REGEXP_EXTRACT()`	패턴 추출
`REGEXP_CONTAINS()`	패턴 존재 확인

Problem :

- 자주 쓰는 문법은 외웠는데 아직 익숙하지 않은 문법들은 아직 힘들다..
- 추가 피쳐 만드는 부분은 어떤식으로 계산 해야하는지 떠오르긴 한데 쿼리문 작성으로 이어지지가 않는다.
- VScode 환경보다 빅쿼리 환경이 아직 많이 불편하다

Try : 쿼리문 퀴즈를 매일 반복 연습이 필요하다